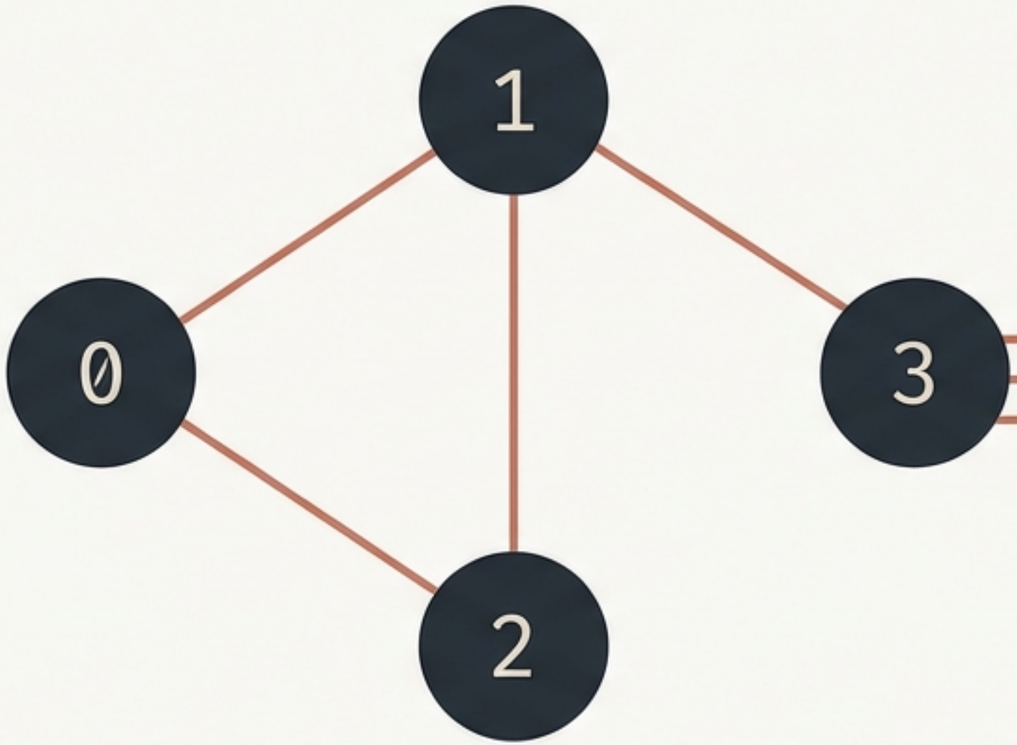
Graph Adjacency List Representation in Python

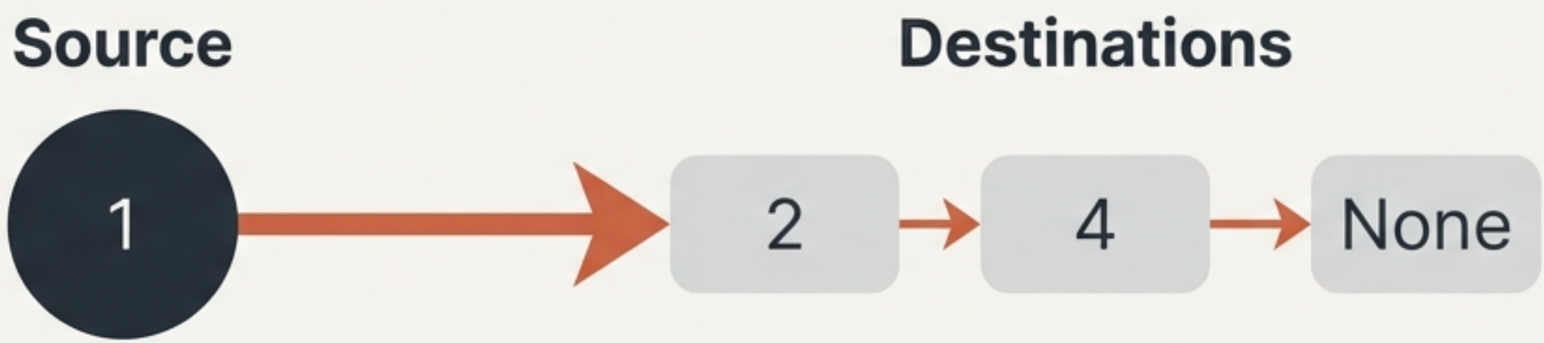
Bridging theoretical computer science with dynamic Python implementation.




```
{
    0: [1, 2],
    1: [0, 2, 3],
    2: [0, 1],
    3: [1]
}
```

Mapping Vertices to their Destinations

- An Adjacency List eliminates empty space by only recording actual connections.
- Every vertex in the graph acts as a Source.
- The source points to a linked list of its directly connected Destinations.

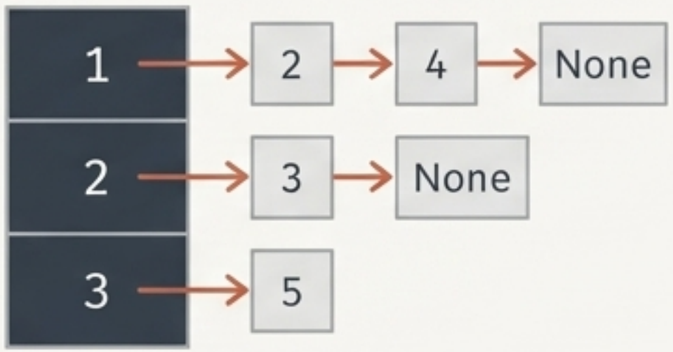


- The source points to a **linked list** of its directly connected Destinations.

Language-Specific Data Structures

In Python, we use a Dictionary where the Key is the Vertex and the Value is a List containing all connected vertices.

C++



Array of Pointers

Java

Key	Value
'A'	[B, C]
'B'	[A, D]
'C'	[D, E]

Maps

Python

```
{ Key : [List] }
   1 : [2, 4],
   2 : [3],
   4 : []
}
```

Dictionaries

Adapting the Data Structure for Edge Direction

Undirected graphs require two simultaneous dictionary updates for every single edge connection.

Directed Graph

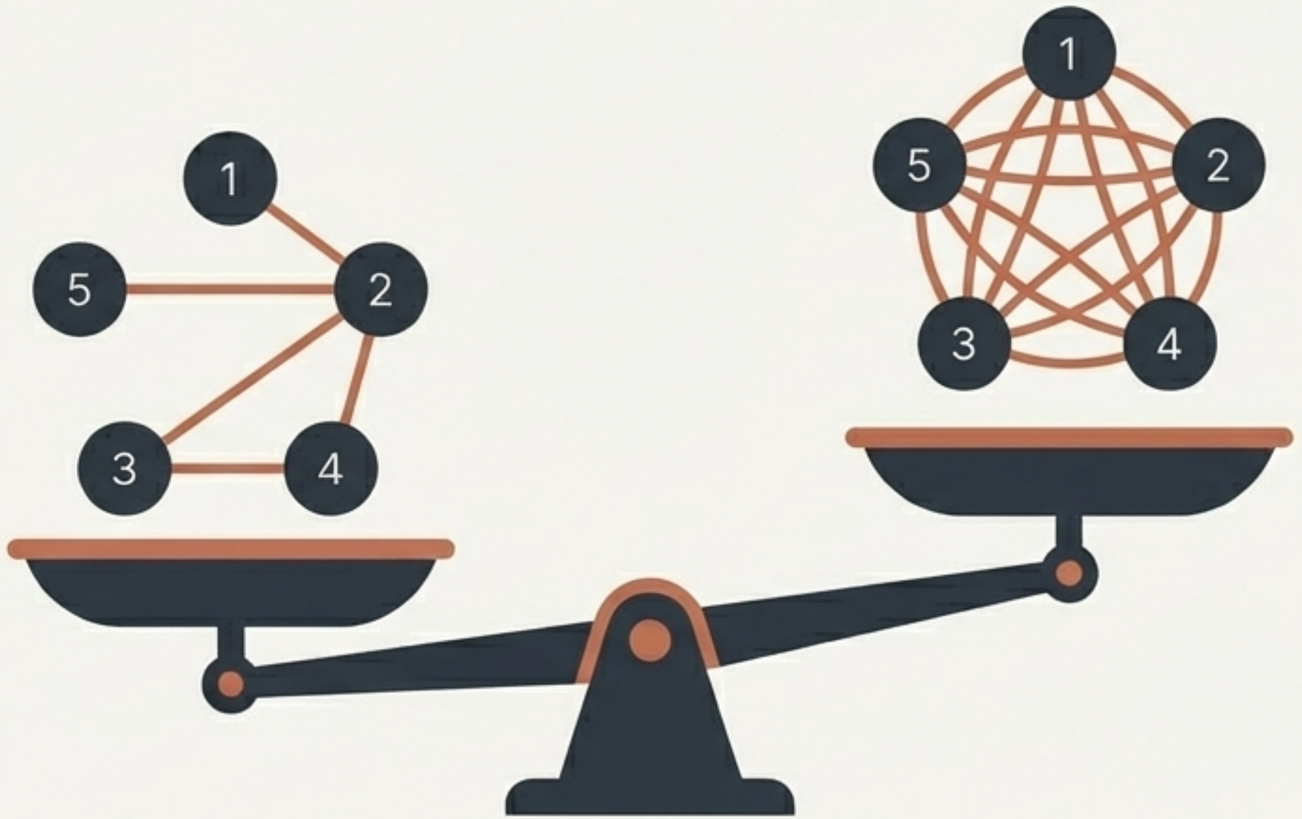

```
{1: [2]}
```

Undirected Graph


```
{1: [2], 2: [1]}
```

Architectural Trade-offs: Sparse vs. Dense

Adjacency Lists are ideal for Sparse Graphs. They allow you to dynamically add or remove vertices without rebuilding the entire structure. Dense graphs generate massive linked lists, negating the memory benefits.



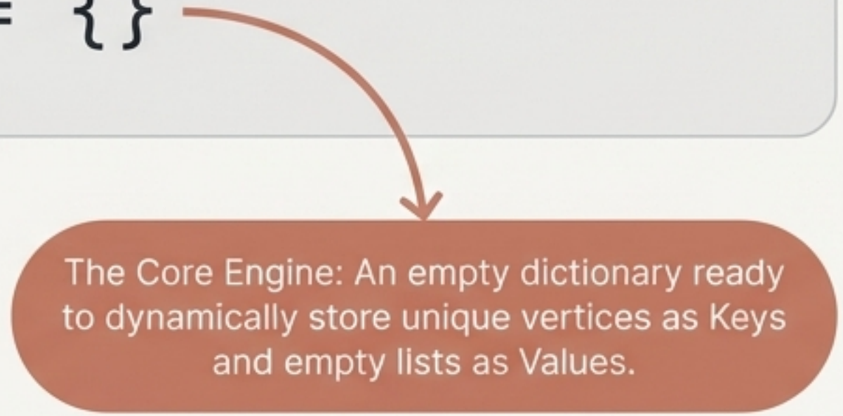
Adjacency List
(Zero memory waste, dynamic scaling)

Adjacency Matrix
(Better performance, fixed size)

Step 1: Initializing the Dynamic Foundation

The Core Engine: An empty dictionary ready to dynamically store unique vertices as Keys and empty lists as Values.

```
class Graph:
    def __init__(self):
        self.adjacency_list = {}
```



Step 2: Safely Adding Unique Vertices

The Uniqueness Guard. Two vertices cannot share the same name.

Initializes the vertex with an empty destination list, ready to receive edges.

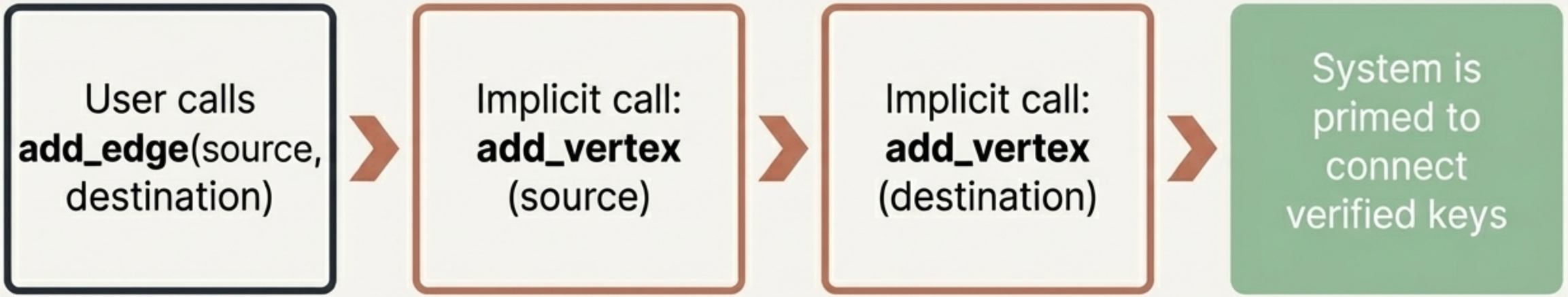
```
def add_vertex(self, vertex):
    if vertex not in self.adjacency_list:
        self.adjacency_list[vertex] = []
```

The Uniqueness Guard. Two vertices cannot share the same name.

Initializes the vertex with an empty destination list, ready to receive edges.

Step 3: Preparing to Connect Vertices

By implicitly calling `add_vertex` inside the edge logic, the graph dynamically builds its missing components on the fly without breaking.



Step 4: Executing the Edge Connection

Locates the specific dictionary Key and adds the destination to the corresponding Value list.

```
# For Directed Graphs
self.adjacency_list[source].append(destination)

# Add this for Undirected Graphs
self.adjacency_list[destination].append(source)
```

Locates the specific dictionary Key.

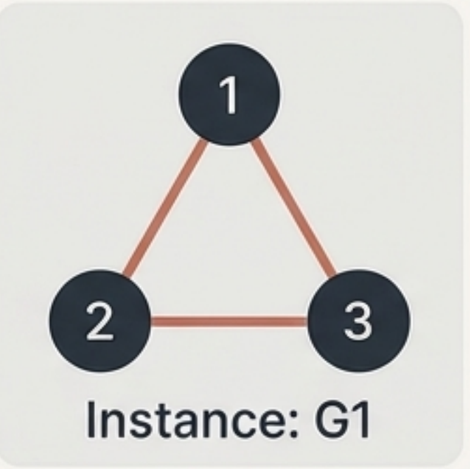
Adds the destination to the corresponding Value list.

Scaling with Object-Oriented Design

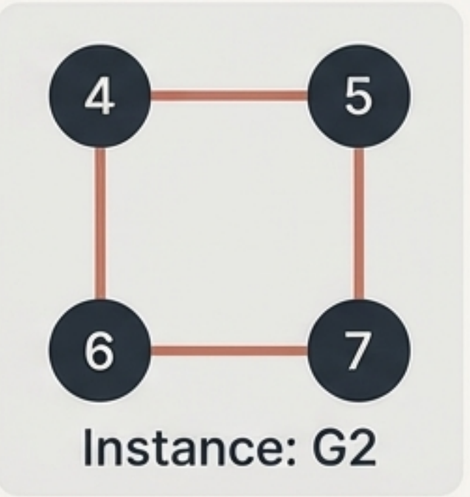
Passing `self` ensures the underlying dictionary is bound to a specific object instance.

This allows a developer to construct G1 and G2 simultaneously in memory without their vertex data colliding or overwriting each other.

Graph Class Blueprint



Instance: G1

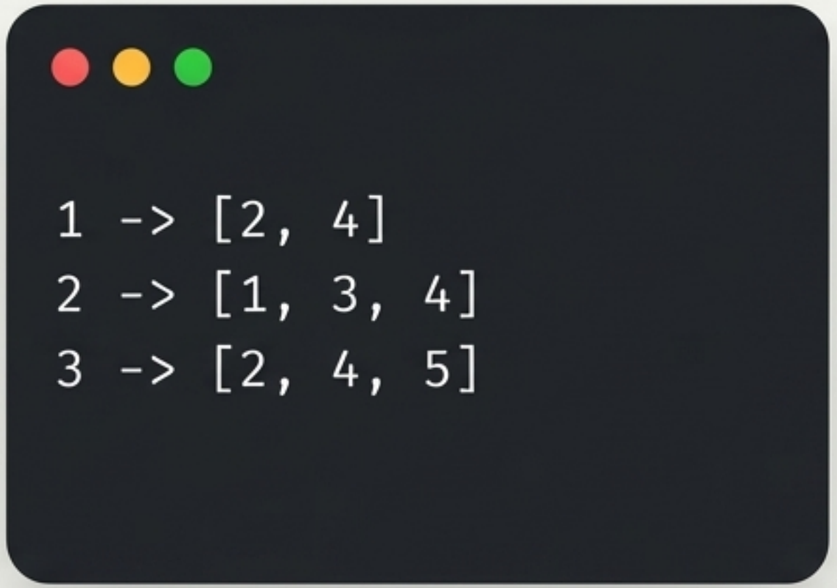


Instance: G2

Generating Human-Readable Output

Iterating directly through the dictionary keys allows us to neatly print the source vertex followed by its array of destinations.

```
for vertex in self.adjacency_list:
    print(vertex, '->', self.adjacency_list[vertex])
```



Extracting Analytical Insights: Vertex Degrees

In a Graph, the number of edges connected to a vertex is its Degree.

Using an Adjacency List, finding the degree of an undirected vertex is highly efficient: it is exactly equal to the length of its destination array.


```
len([1, 2, 3, 5]) = 4
```

Degree

Adjacency List Reference Guide

The Structure

```
{ Source_Vertex : [Destination_1, Destination_2] }
```

The Rules

Dictionary keys mandate unique vertices.
Directed graphs append once;
Undirected append twice.

The Ideal Use-Case

Sparse graphs (few edges) where dynamic addition/removal of nodes is required and memory waste must be avoided.